

VZ200/VZ300 Floppy Disk transfer kit

Purpose

The purpose of this document is to capture the design of a low cost floppy disk to sd card transfer mechanism. This document will:

1. Explain the objective of the project
2. List the minimal hardware required to achieve floppy disk transfer
3. Document an example design and explain how it works

Objective

The desired goal is to give disk drive owners a fast, simple, reliable mechanism to preserve the contents of a physical VZ floppy disk. The resulting file will be a “disk image” that can be mapped as an emulated disk in VZEM

Minimal hardware required

There are many solutions to this problem. The solution presented here uses a cheap 5 volt microcontroller, an SD card reader, and two 5 volt dip package logic IC's. Surface mount IC's, and non 5 volt ICs are avoided. The components were chosen to make the kit breadboard friendly, easy to assemble, and allow for modifications.

The hardware required and approximate cost is:

- | | |
|---|-----|
| 1. Arduino nano microcontroller | \$5 |
| 2. 74LS138 line decoder | \$1 |
| 3. 74LS259 latch (or similar) | \$1 |
| 4. MicroSD card reader | \$5 |
| 5. VZ compatible edge connector | \$2 |
| 6. Breadboard/perfboard | \$5 |
| 7. Misc items – capacitors, hookup wire | \$1 |

Total cost: appx \$20

Optional hardware

The hardware specified above will allow one way transfer from the VZ to the microcontroller, which is all that is required to dump a disk image. The kit would be more flexible if the VZ200 could also read data from the microcontroller, allowing for two way communication. A miniature display and pushbuttons would also add flexibility.

Circuit Design

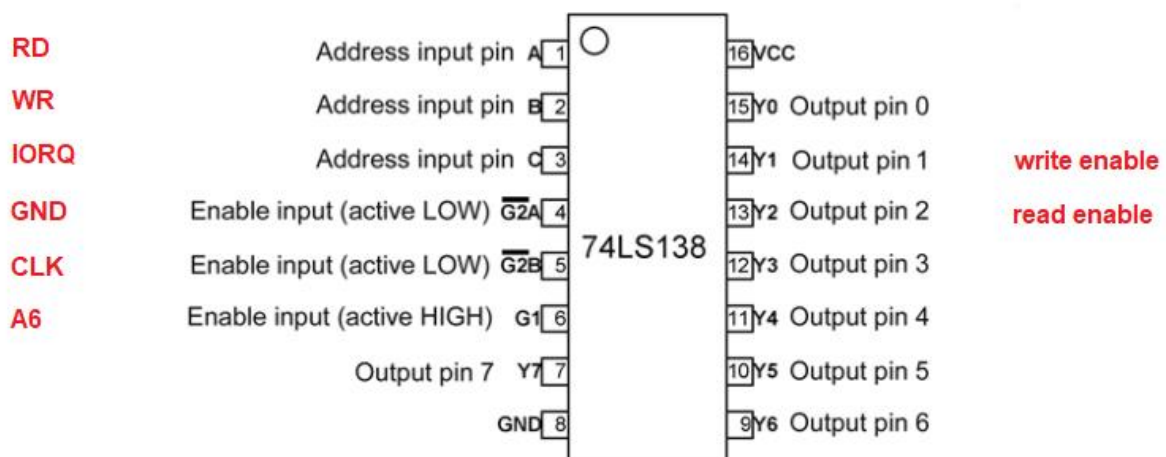
An Arduino nano is not fast enough to sample the control lines of the VZ200 in real time via polling, or respond in time via interrupts. The nano also has a limited number of pins. This is why the supporting IC's 74LS138 decoder and 74LS259 latch are required.

The VZ port used for this circuit is 40h. This port was chosen as it is essentially "unused" by existing hardware standards. The VZ disk drive uses port 20h. Using port 40h allows the disk drive and transfer kit to co-exist. The kit plugs into the piggyback expansion connector of the disk controller, usually used by the ram expansion cartridges.

Software on the VZ200 will read data from the physical floppy disk and write this data to port 40h, where it will be read by the microcontroller and saved to the SD card.

The Z80 control lines for the write operation are /IORQ, /WR, A6

Pin assignment to the 74LS138 is below. When the Z80 executes an OUT instruction to port 40h, WR and IORQ will be low. A6 will be high. This combination will cause Y1, mapped as the "write enable" pin, to go low.



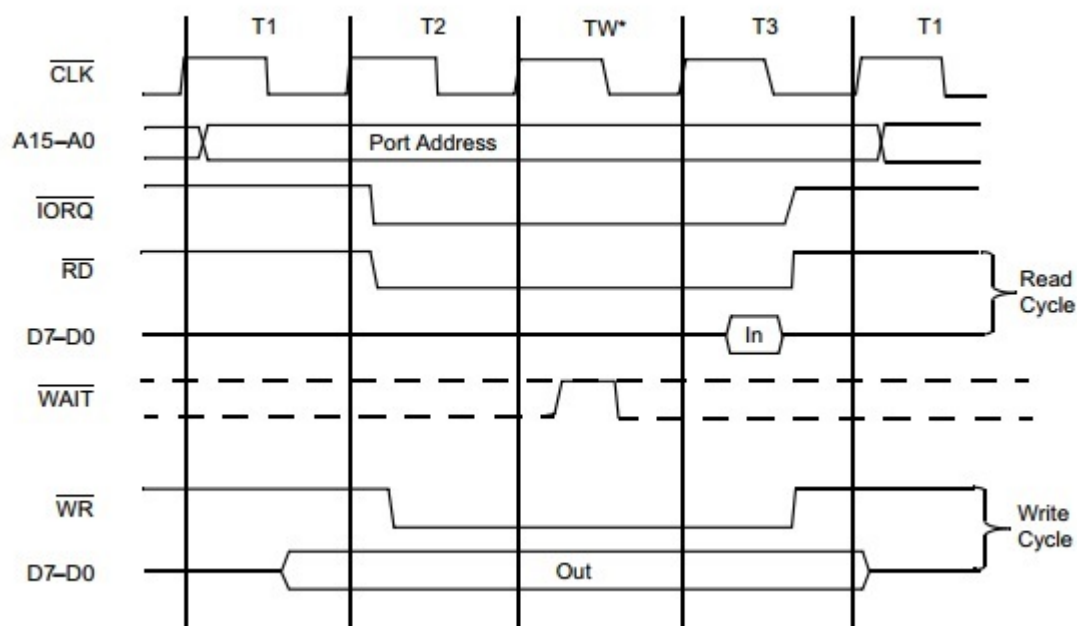
The 74LS138 truth table below shows how the combination of control lines trigger the outputs Y1 and Y2. (Note – Y2 read enable is currently not used, but is there for future expansion)

Inputs		Outputs										
Enable		Select										
G1	G2 (Note 1)	C	B	A	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
X	H	X	X	X	H	H	H	H	H	H	H	H
L	X	X	X	X	H	H	H	H	H	H	H	H
H	L	L	L	L	L	H	H	H	H	H	H	H
H	L	L	L	H	H	L	H	H	H	H	H	H
H	L	L	H	L	H	H	L	H	H	H	H	H
H	L	L	H	H	H	H	H	L	H	H	H	H
H	L	H	L	L	H	H	H	H	L	H	H	H
H	L	H	L	H	H	H	H	H	H	L	H	H
H	L	H	H	L	H	H	H	H	H	H	L	H
H	L	H	H	H	H	H	H	H	H	H	H	L

WR
RD

A6 IORQ
G2A=GND WR
G2B=CLK RD

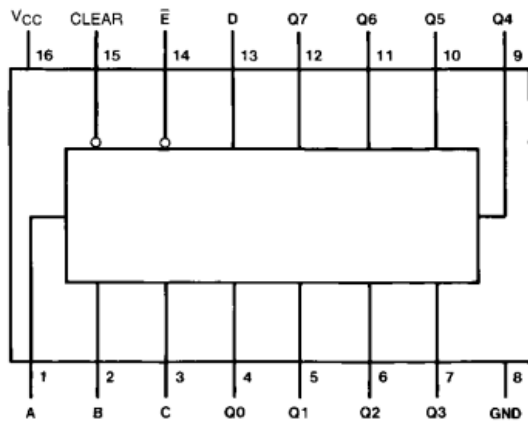
The Z80 CLK is one of the inputs. When WR and IORQ go low the Z80 is in state T2 with the clock high. By using a low CLK as an input, Y1 will not be enabled until the second half of T2 leading into TW, which is the critical timing point for a data write.



Output Y1 is wired to the /E (enable) pin of the 74LS259. The data pin D is wired to Z80 D7. When /E is low, the contents of Z80 D7 are latched to Q0.

Q0 is selected by setting inputs A, B & C to GND. The CLEAR pin is permanently set high by wiring to the 5v line via a 10k pullup resistor.

Connection Diagram



Z80 D7 = D
LS138 Y1 = /E

Function Table

Inputs		Output of Addressed Latch	Each Other Output	Function
Clear	$\overline{E}$			
H	L	D	Q_{i0}	Addressable Latch
H	H	Q_{i0}	Q_{i0}	Memory
L	L	D	L	8-Line Demultiplexer
L	H	L	L	Clear

Latch Selection Table

Select Inputs			Latch Addressed
C	B	A	
L	L	L	0
L	L	H	1
L	H	L	2
L	H	H	3
H	L	L	4
H	L	H	5
H	H	L	6
H	H	H	7

A better choice of latch IC for this project is the 74LS273, but I don't have any of these in my junk box at the time of writing 😊

The 74LS259 and close alternatives allows for a single data bit to be latched. A single data bit is the simplest solution. It would be possible to use another latch IC that allows for all eight Z80 data lines.

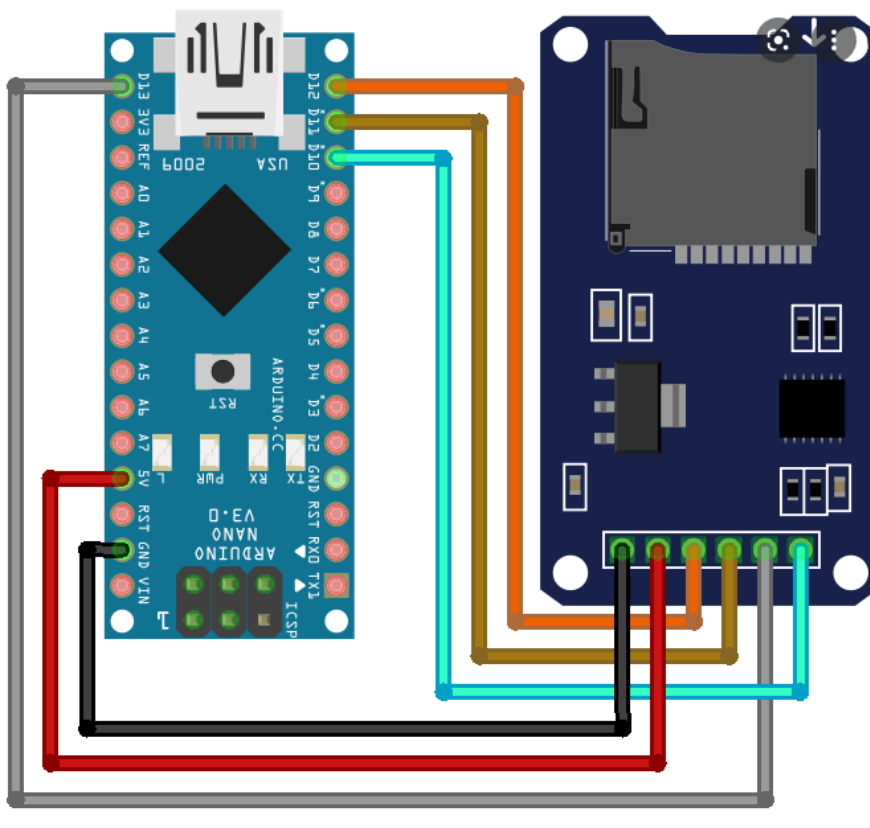
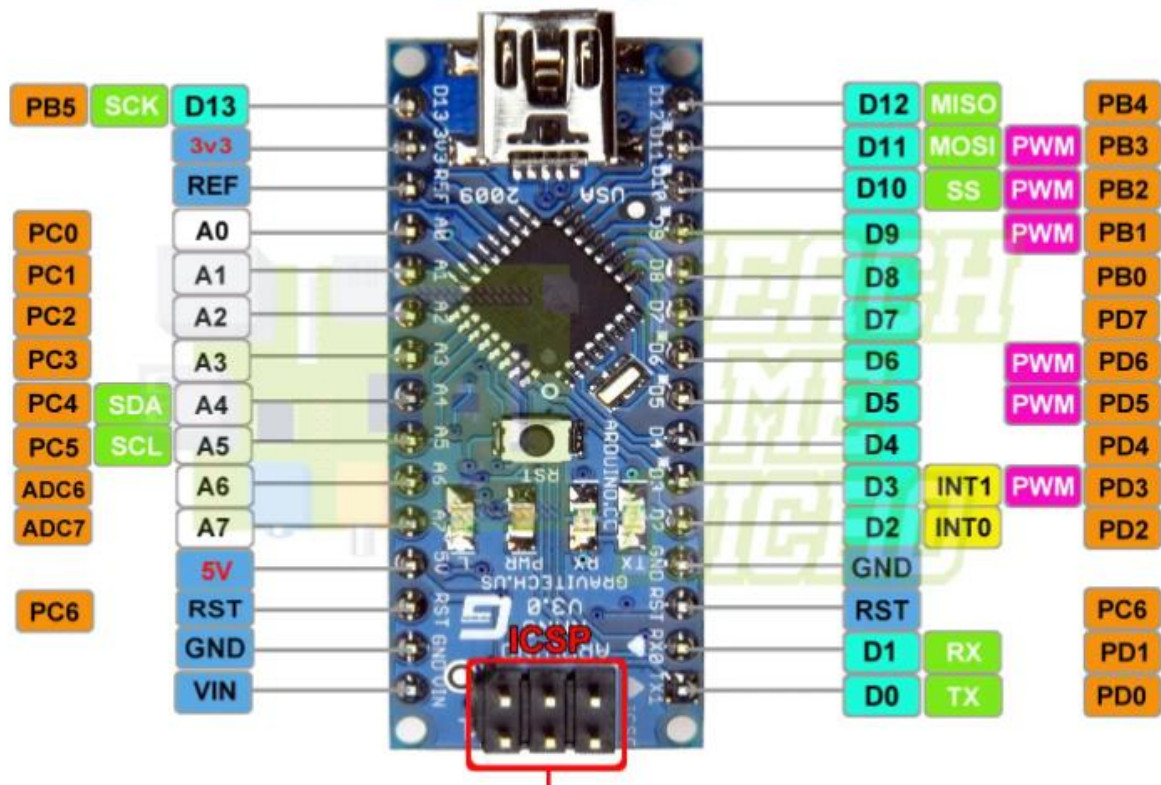
Output Y1 is also wired to the Arduino nano pin D2, which is configured in software as an interrupt pin. When D2 goes low via Y1, an interrupt routine will be called that reads the bit latched on Q0.

Arduino nano pin D4 (PD4) is wired to Q0 to read the bit

On the Z80 side, a byte is written by sequentially rotating each bit through D7. A software delay is used to give the microcontroller enough time to read each bit, reconstruct the byte, and write to the SD card.

The SD wiring diagram is shown below

Mini USB Port

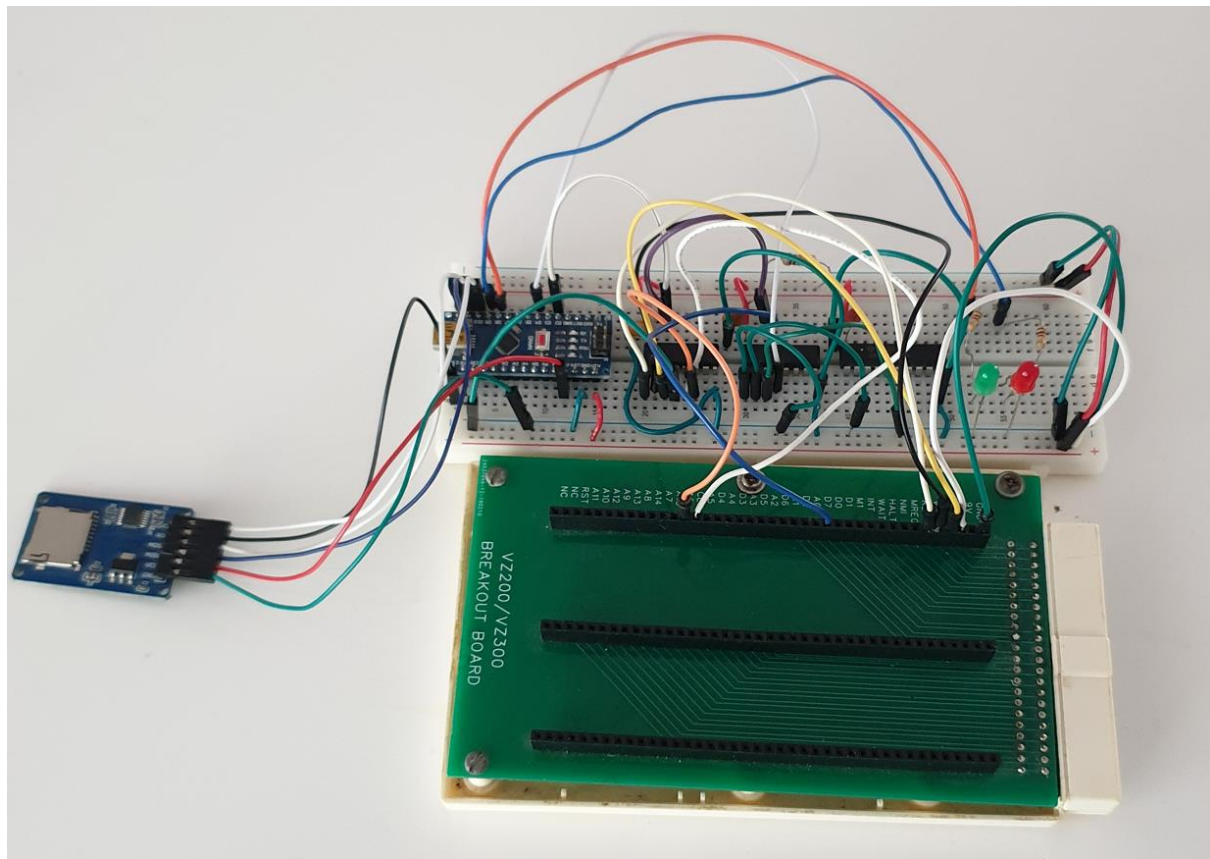


Example build on a breadboard. Two led's were connected to the Arduino to provide status:

- On powerup, if an sd card is detected the green led will light. If no sd card is detected the red led will light
- The red led will light as data is being written to the sd card

Refer to the Arduino sketch for additional information on pin assignment

A tft display with sd card could optionally be connected to the Arduino, displaying instructions/progress bar etc



Prototype plugged into disk controller piggyback edge connector

